



DCA0305

ivanovitch.silva@ufrn.br

Machine Learning Based Systems Design

Rethinking the training loop: a simple classification problem

- ▼ Part I: Fundamentals
 - › Chapter 0: Visualizing Gradient Descent
 - › Chapter 1: A Simple Regression Problem
 - › Chapter 2: Rethinking the Training Loop
 - › Chapter 2.1: Going Classy
 - › Chapter 3: A Simple Classification Problem
 - › Part II: Computer Vision
 - › Part III: Sequences
 - › Part IV: Natural Language Processing



Going Classy

Object-Oriented Programming – OOP

lesson03a.ipynb, lesson03b.ipynb

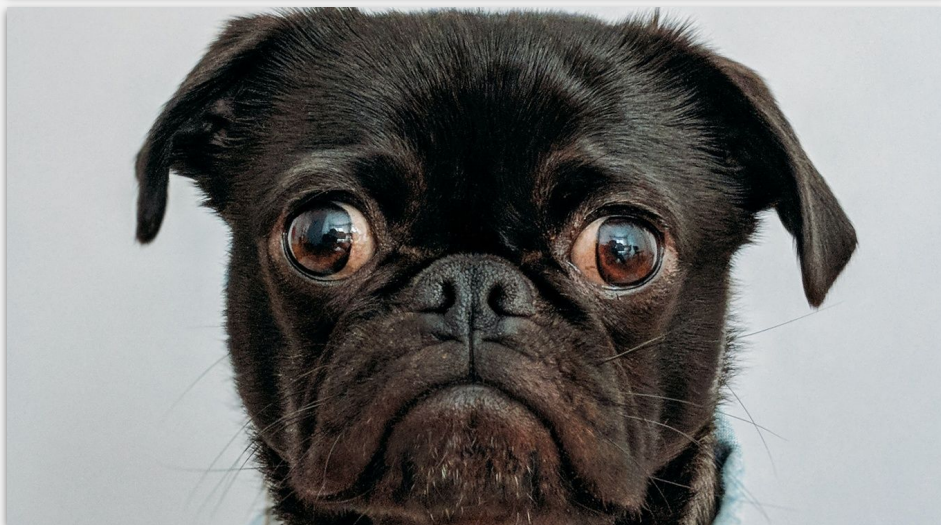
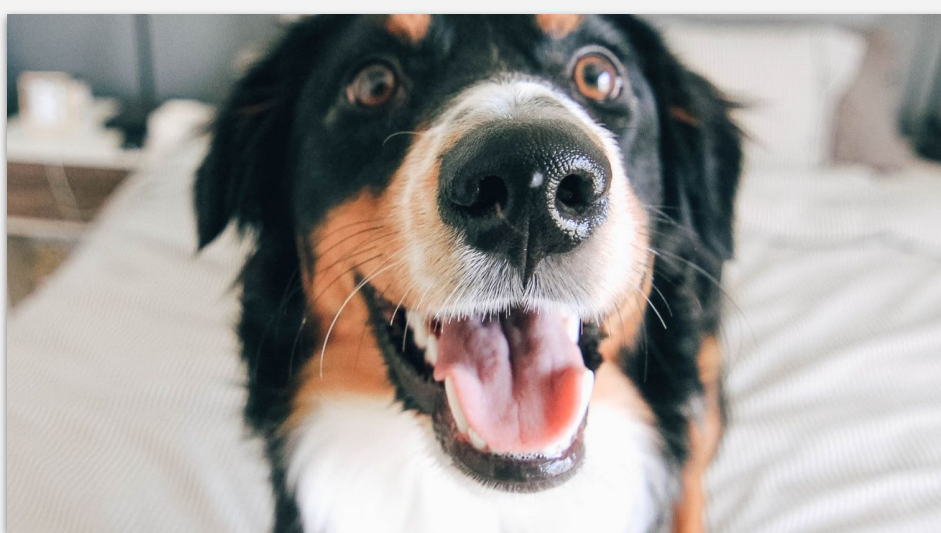
```
139 title="Home"
140 target=""
141 rule="no"
142 href={url}
143 >
144 </a>
145 </li>
146 </ul>
147 </div>
148 </div>
149 </div>
150 </div>
151 </div>
152 </div>
153 </div>
154 </div>
155 </div>
156 </div>
157 </div>
158 </div>
159 </div>
160 </div>
161 </div>
162 </div>
163 </div>
164 </div>
165 </div>
166 </div>
167 </div>
168 </div>
169 </div>
170 </div>
171 </div>
172 </div>
173 </div>
174 </div>
175 </div>
176 </div>
177 </div>
178 </div>
179 </div>
180 </div>
181 </div>
182 </div>
183 </div>
184 </div>
185 </div>
186 </div>
187 </div>
188 </div>
189 </div>
190 </div>
191 </div>
192 </div>
193 </div>
194 </div>
195 </div>
196 </div>
197 </div>
198 </div>
199 </div>
200 </div>
201 </div>
202 </div>
203 </div>
204 </div>
205 </div>
206 </div>
207 </div>
208 </div>
209 </div>
```

Attribute	Description
<code>model</code>	Neural network model (PyTorch object)
<code>loss_fn</code>	Loss function
<code>optimizer</code>	Optimizer (e.g., SGD, Adam)
<code>device</code>	Execution device (<code>cuda</code> or <code>cpu</code>)
<code>train_loader</code>	DataLoader for the training dataset
<code>val_loader</code>	DataLoader for the validation dataset
<code>losses</code>	List of training losses per epoch
<code>val_losses</code>	List of validation losses per epoch
<code>total_epochs</code>	Total number of completed training epochs
<code>train_step_fn</code>	Internal function to execute a training step
<code>val_step_fn</code>	Internal function to execute a validation step

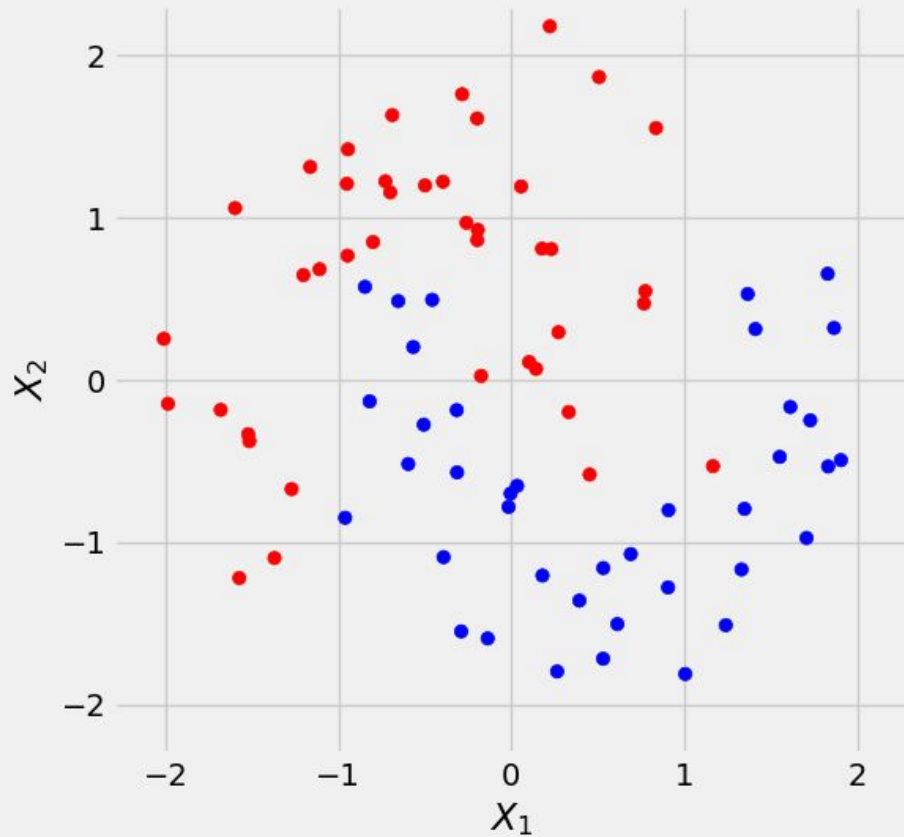
Method	Description
<code>__init__(model, loss_fn, optimizer)</code>	Constructor. Initializes all attributes
<code>to(device)</code>	Sets the device and moves the model to it
<code>set_loaders(train_loader, val_loader=None)</code>	Assigns train and validation DataLoaders
<code>_make_train_step_fn()</code>	Creates the training step function
<code>_make_val_step_fn()</code>	Creates the validation step function
<code>_mini_batch(validation=False)</code>	Executes one mini-batch loop for training or validation
<code>set_seed(seed=42)</code>	Sets random seeds for reproducibility
<code>train(n_epochs, seed=42)</code>	Executes the training loop for <code>n_epochs</code>
<code>save_checkpoint(filename)</code>	Saves the model and optimizer state to a file
<code>load_checkpoint(filename)</code>	Loads model and optimizer state from a file
<code>predict(x)</code>	Predicts the output for input <code>x</code>
<code>plot_losses()</code>	Plots the training and validation loss curves

A simple Classification Problem

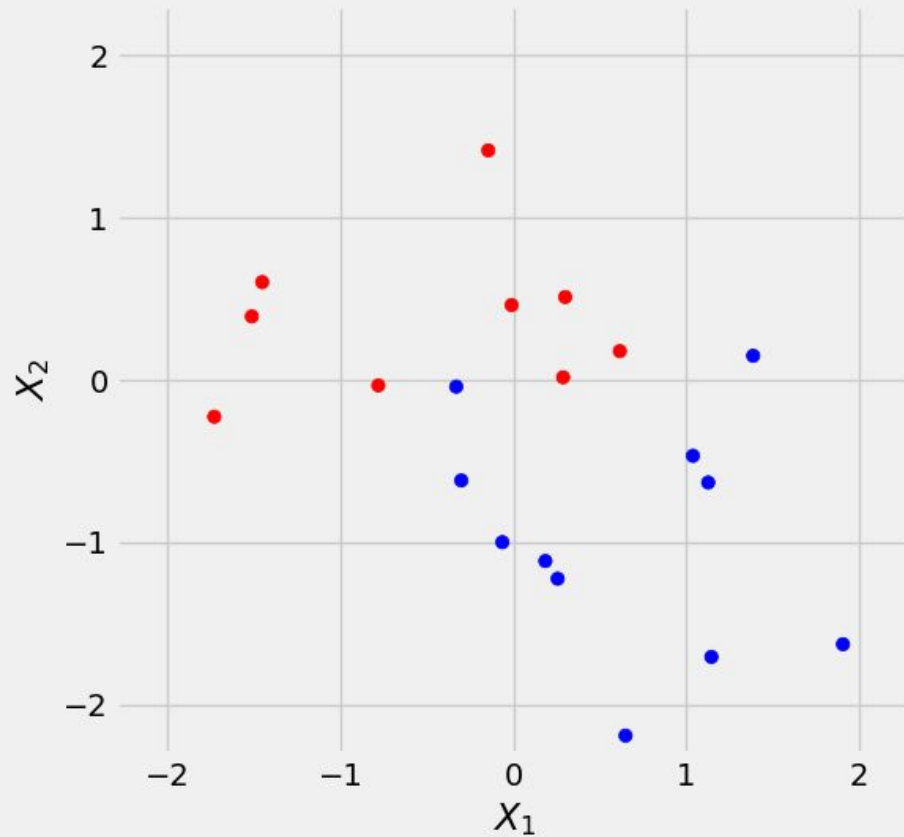
week04c.ipynb



Generated Data - Train



Generated Data - Validation





```
# Generate a synthetic two-dimensional dataset with 100 samples
# 'make_moons' is used for binary classification problems
X, y = make_moons(n_samples=100, noise=0.3, random_state=0)

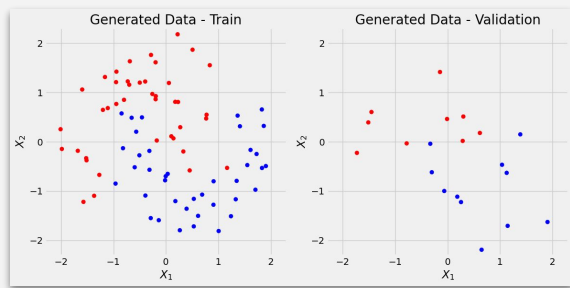
# Split the dataset into training and validation sets
# 20% of the data is used for validation
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=.2, random_state=13)

# Initialize a StandardScaler instance
sc = StandardScaler()

# Fit the scaler only on the training data
# This computes the mean and standard deviation to be used for later scaling
sc.fit(X_train)

# Transform both training and validation sets
# Scale the training data
X_train = sc.transform(X_train)

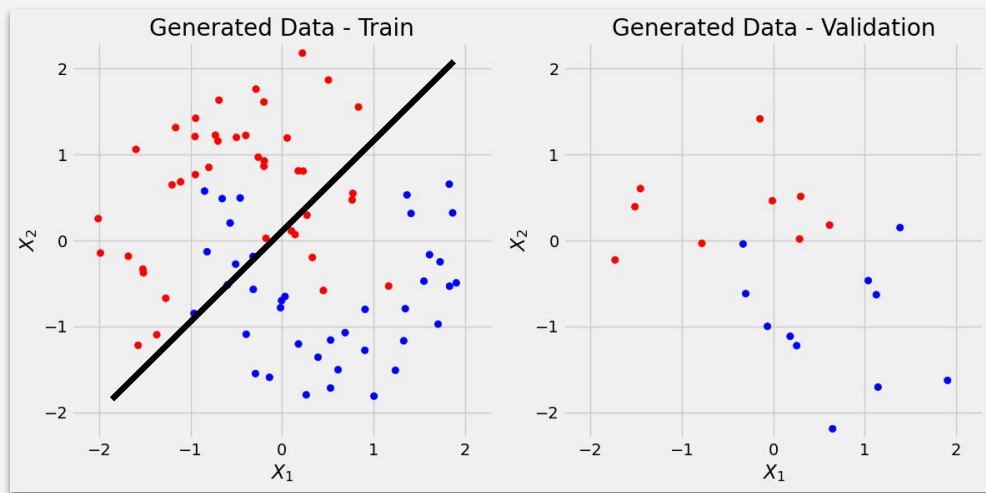
# Apply the same transformation to the validation data
X_val = sc.transform(X_val)
```

Linear Regression vs Logistic Regression

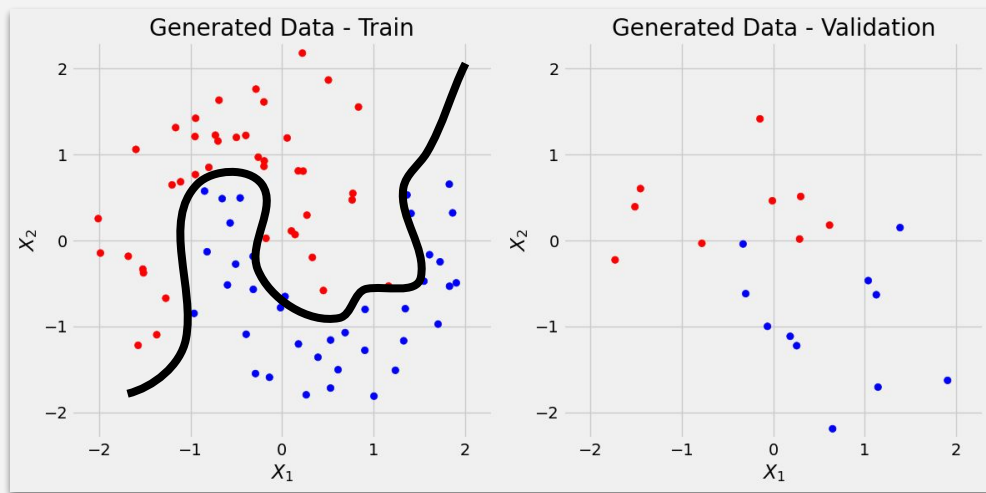
The **dependent variable** is a metric variable: eg. height, speed, salary, grade, etc

The **dependent variable** is a dichotomous variable: eg. red or blue, happy or sad, 0 or 1, gender, etc

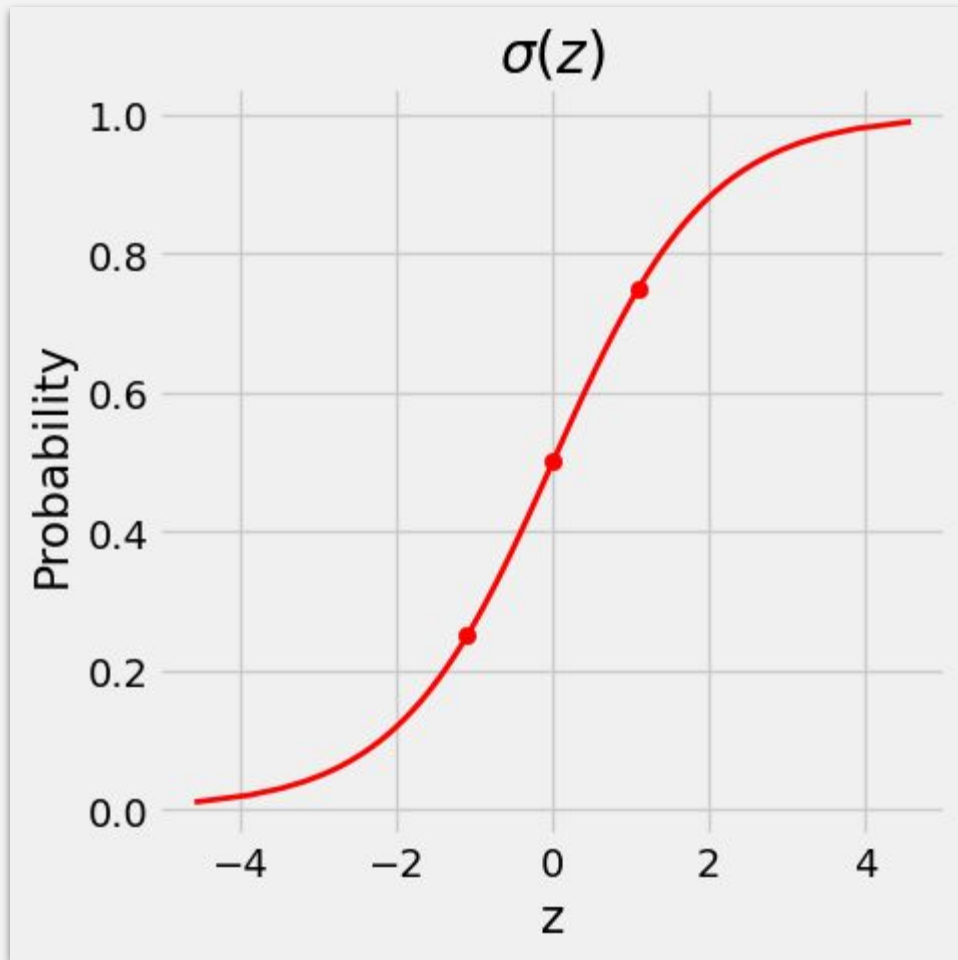


$$y = b + w_1x_1 + w_2x_2 + \epsilon$$

$$y = \begin{cases} 1, & \text{if } b + w_1x_1 + w_2x_2 \geq 0 \\ 0, & \text{if } b + w_1x_1 + w_2x_2 < 0 \end{cases}$$



Mapping a linear regression model to **discrete labels**.



Only apply linear regression is not enough. The goal of the logistic regression is to estimate the **probability of occurrence**.

Logits

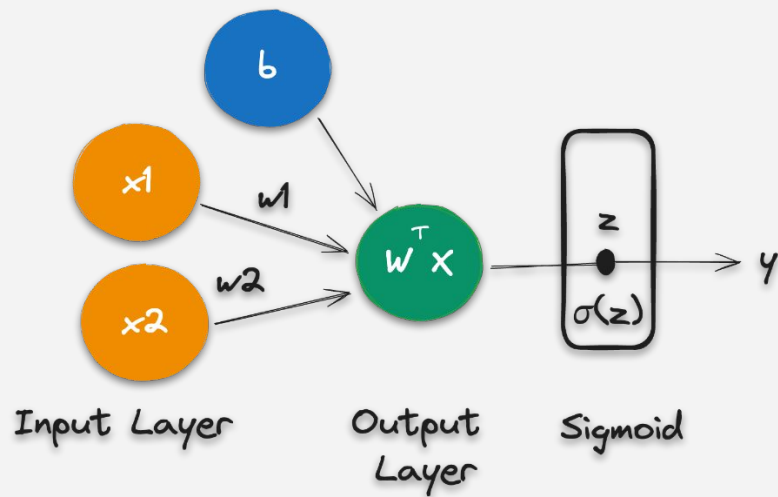
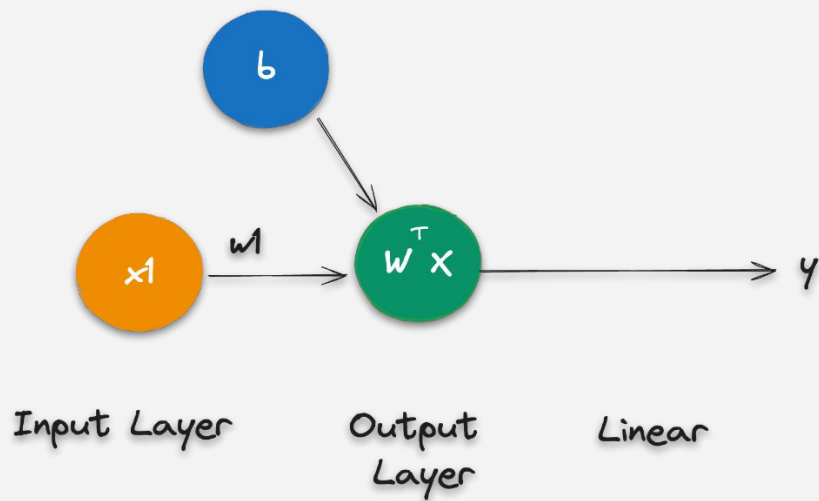
$$z = b + w_1x_1 + w_2x_2$$

$$P(y = 1) \approx 1.0, \text{ if } z \gg 0$$

$$P(y = 1) = 0.5, \text{ if } z = 0$$

$$P(y = 1) \approx 0.0, \text{ if } z \ll 0$$

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

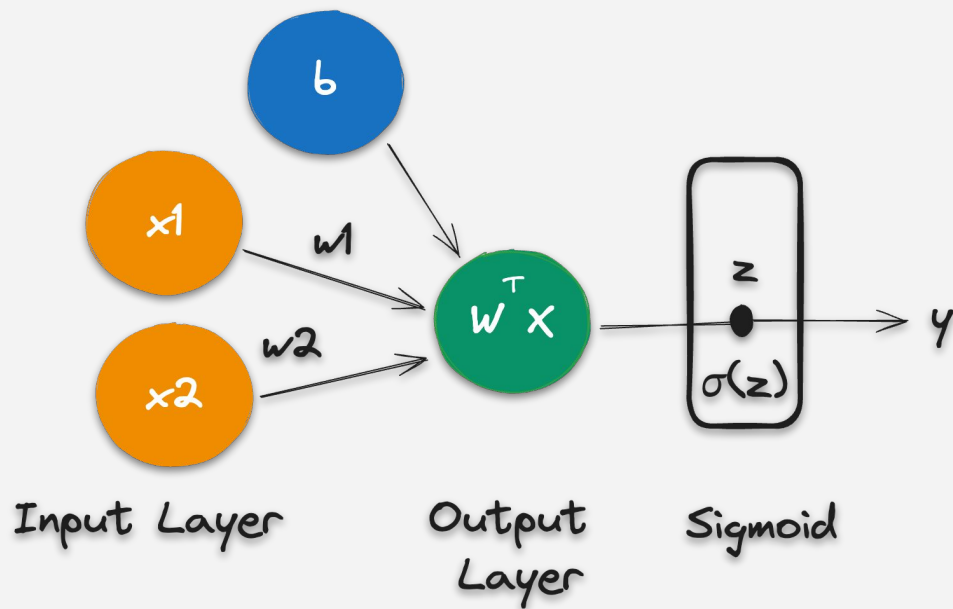


Linear Regression vs Logistic Regression

Logistic Regression

A Note on Notation

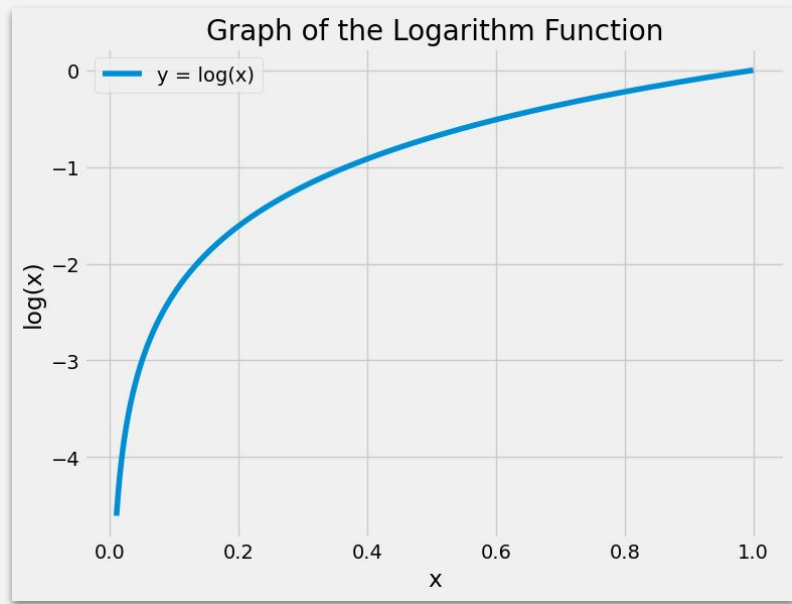
$$W = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}_{(3 \times 1)}; X = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}_{(3 \times 1)}$$



$$\begin{aligned} z &= W^T X = \begin{bmatrix} - & w^T & - \end{bmatrix}_{(1 \times 3)} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}_{(3 \times 1)} = \begin{bmatrix} b & w_1 & w_2 \end{bmatrix}_{(1 \times 3)} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}_{(3 \times 1)} \\ &= b + w_1 x_1 + w_2 x_2 \end{aligned}$$

```
torch.manual_seed(42)
model = nn.Sequential()
model.add_module('linear', nn.Linear(2, 1))
model.add_module('sigmoid', nn.Sigmoid())
print(model.state_dict())

OrderedDict([('linear.weight', tensor([[0.5406,
0.5869]])), ('linear.bias', tensor([-0.1657]))])
```



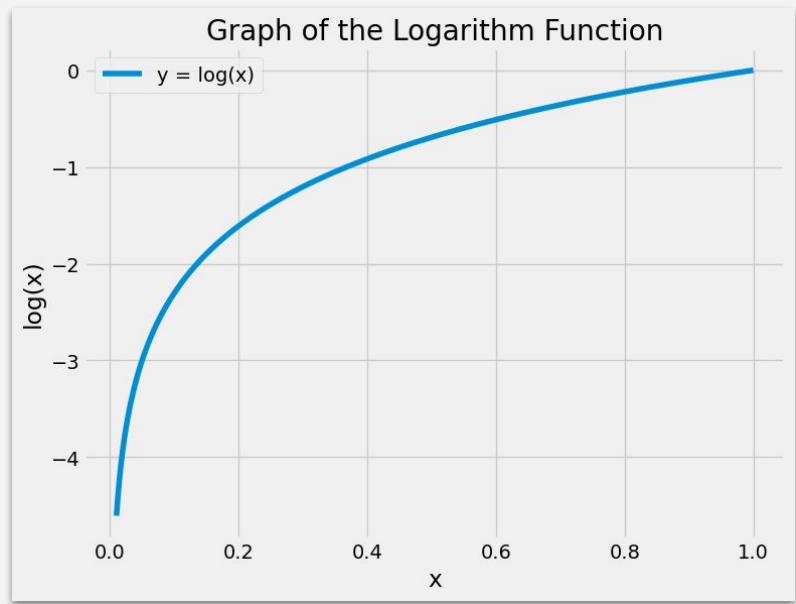
#intuitive-way

We already have a model,
and now we need to define
an appropriate **loss** for the
logistic regression.

$$y_i = 1 \Rightarrow error_i = \log(P(y_i = 1))$$

$$P(y_i = 0) = 1 - P(y_i = 1)$$

$$y_i = 0 \Rightarrow error_i = \log(1 - P(y_i = 1))$$

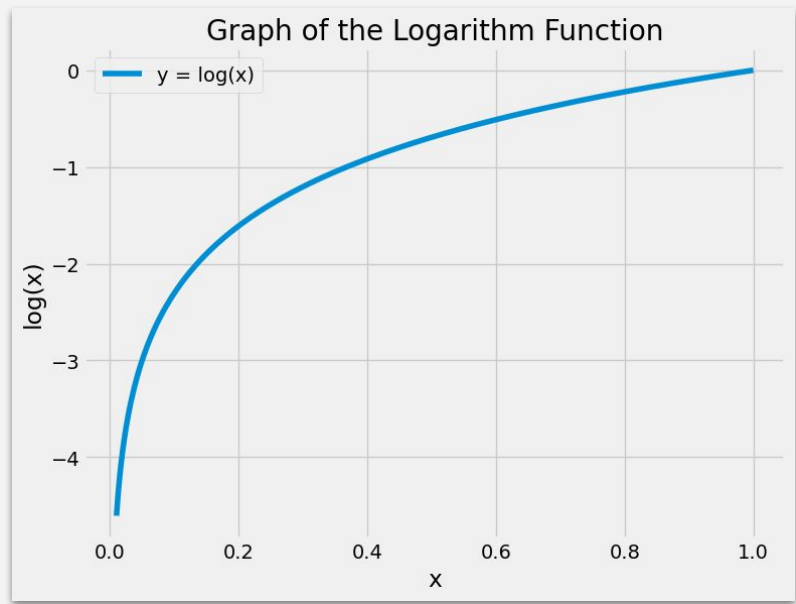


#intuitive-way

We already have a model, and now we need to define an appropriate **loss** for the logistic regression.

Binary Cross-Entropy (BCE)

$$BCE(y) = -\frac{1}{(N_{\text{pos}} + N_{\text{neg}})} \left[\sum_{i=1}^{N_{\text{pos}}} \log(P(y_i = 1)) + \sum_{i=1}^{N_{\text{neg}}} \log(1 - P(y_i = 1)) \right]$$



#intuitive-way

We already have a model, and now we need to define an appropriate **loss** for the logistic regression.

Binary Cross-Entropy (BCE)

$$BCE(y) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(P(y_i = 1)) + (1 - y_i) \log(1 - P(y_i = 1))]$$

Binary Cross-Entropy (BCE)

$$BCE(y) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(P(y_i = 1)) + (1 - y_i) \log(1 - P(y_i = 1))]$$

```
loss_fn = nn.BCELoss(reduction='mean')

dummy_labels = torch.tensor([1.0, 0.0])
dummy_predictions = torch.tensor([.9, .2])

# RIGHT
right_loss = loss_fn(dummy_predictions, dummy_labels)

# WRONG
wrong_loss = loss_fn(dummy_labels, dummy_predictions)

print(right_loss, wrong_loss)
tensor(0.1643) tensor(15.0000)
```

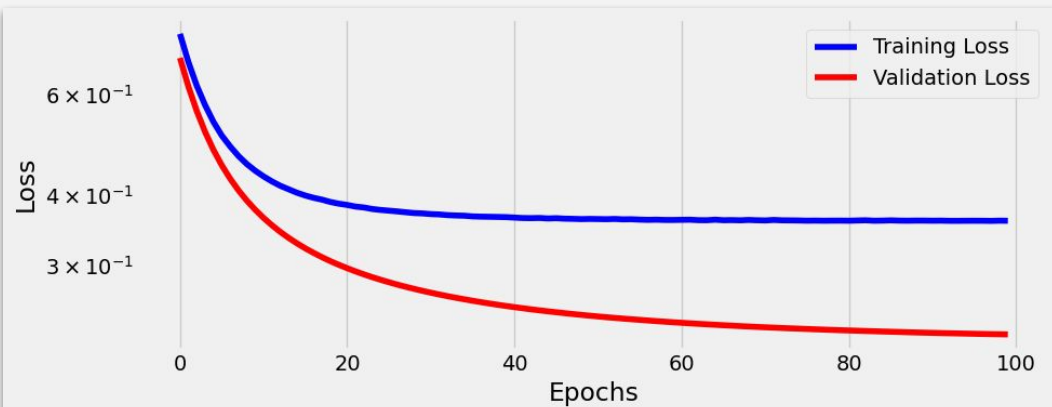
Model Configuration

```
# Sets learning rate - this is "eta" ~ the "n" like Greek letter
lr = 0.1

torch.manual_seed(42)
model = nn.Sequential()
model.add_module('linear', nn.Linear(2, 1))

# Defines a SGD optimizer to update the parameters
optimizer = optim.SGD(model.parameters(), lr=lr)

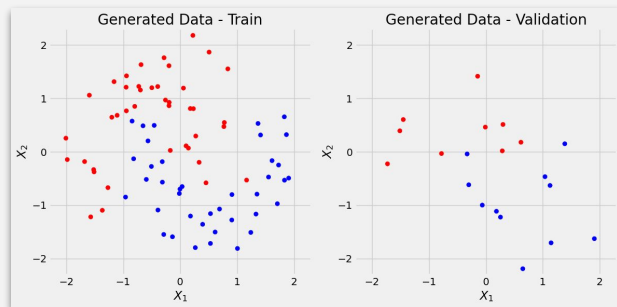
# Defines a BCE loss function
loss_fn = nn.BCEWithLogitsLoss()
```



Training

```
n_epochs = 100

arch = Architecture(model, loss_fn, optimizer)
arch.set_loaders(train_loader, val_loader)
arch.set_seed(42)
arch.train(n_epochs)
```



Making Predictions

$$y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

$$y = \begin{cases} 1, & \text{if } P(y = 1) \geq 0.5 \\ 0, & \text{if } P(y = 1) < 0.5 \end{cases}$$

```
# prediction logits (z)
logits_val = arch.predict(X_val[:4])
logits_val

array([[ -0.37522304],
       [  0.7390274 ],
       [-2.5800889 ],
       [-0.93623203]], dtype=float32)
```

```
# prediction probabilities
prob_val = torch.sigmoid(torch.as_tensor(logits_val[:4]).float())
prob_val

tensor([[0.4073],
        [0.6768],
        [0.0704],
        [0.2817]])
```

Decision Boundary

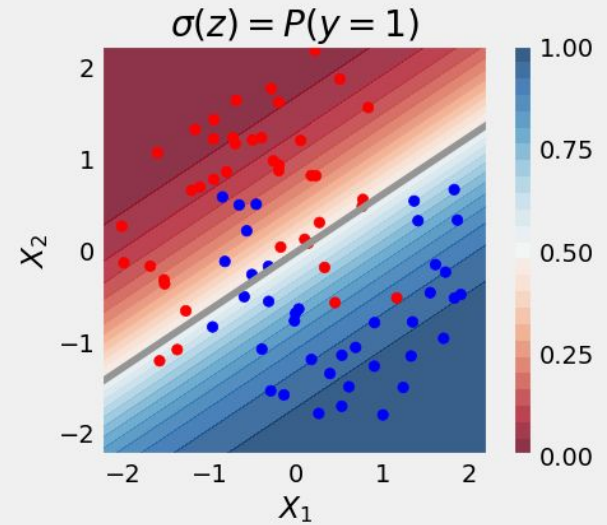
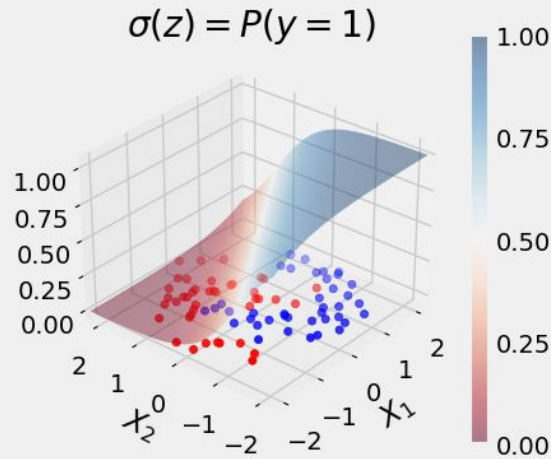
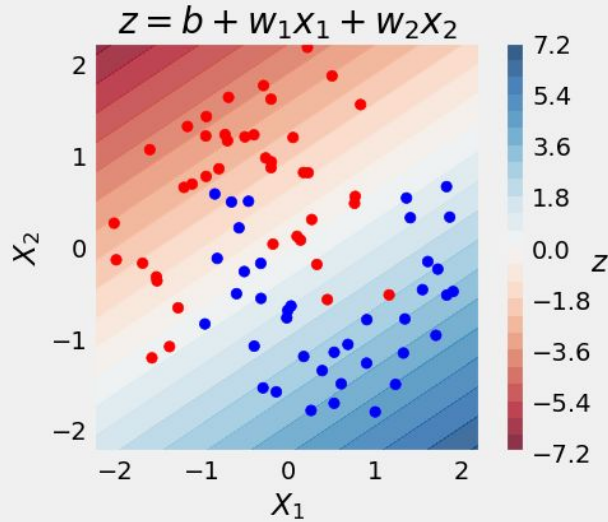
$$\begin{aligned} z &= 0 = b + w_1x_1 + w_2x_2 \\ -w_2x_2 &= b + w_1x_1 \\ x_2 &= -\frac{b}{w_2} - \frac{w_1}{w_2}x_1 \end{aligned}$$

$$\begin{aligned} x_2 &= -\frac{0.0591}{1.8693} + \frac{1.1806}{1.8693}x_1 \\ x_2 &= -0.0316 + 0.6315x_1 \end{aligned}$$

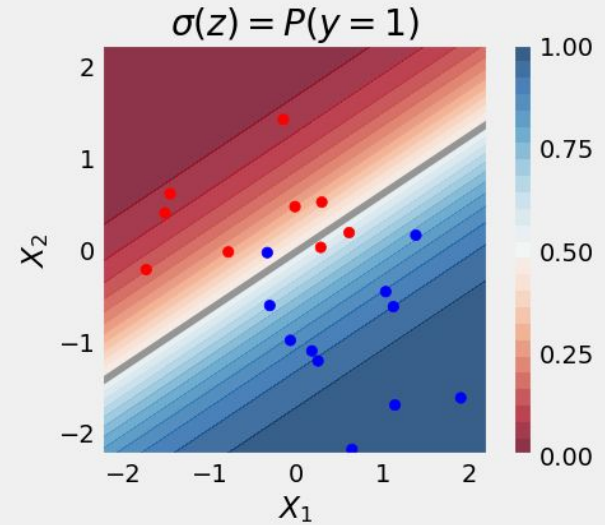
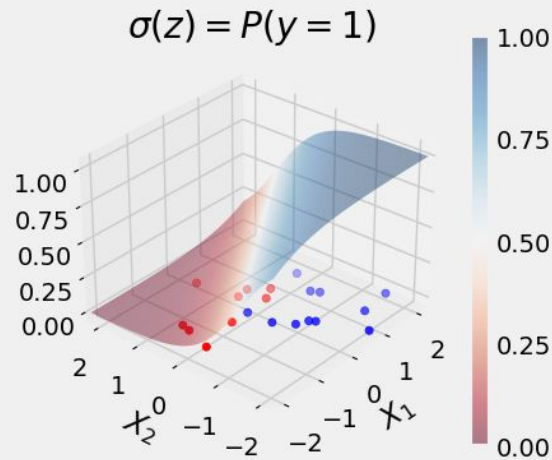
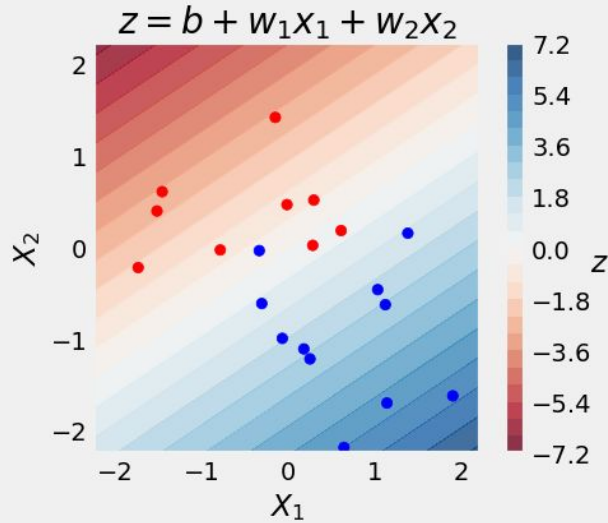
```
print(model.state_dict())  
OrderedDict([('linear.weight', tensor([[ 1.1806, -1.8693]])), ('linear.bias', tensor([-0.0591]))])
```

$$\begin{aligned} z &= b + w_1x_1 + w_2x_2 \\ z &= -0.0591 + 1.1806x_1 - 1.8693x_2 \end{aligned}$$

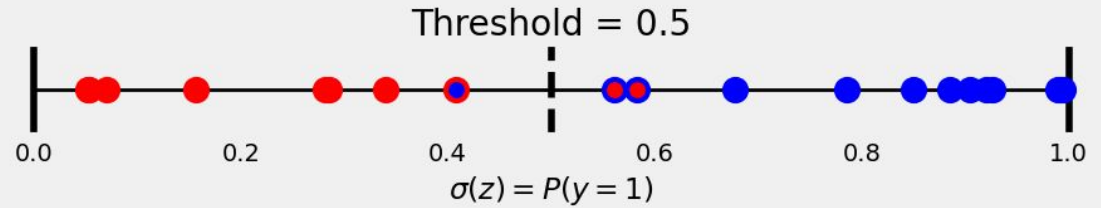
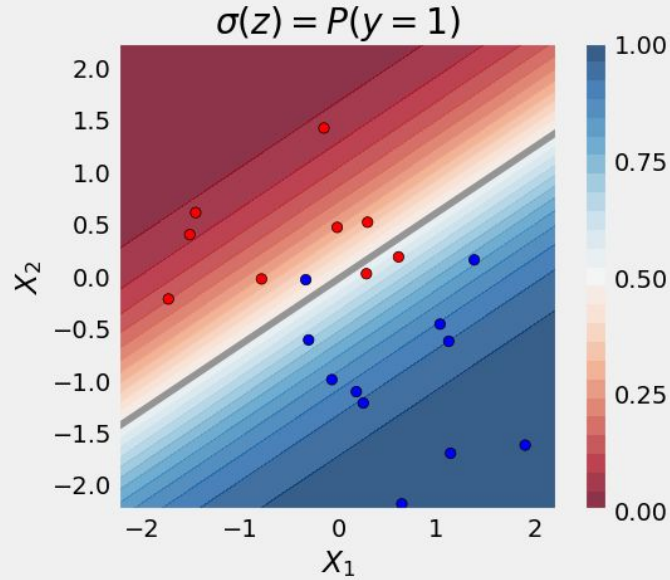
Decision Boundary (training data)



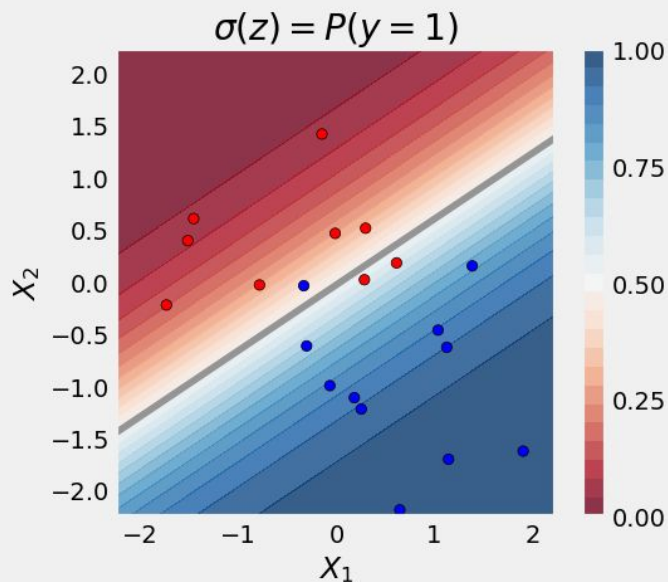
Decision Boundary (validation data)



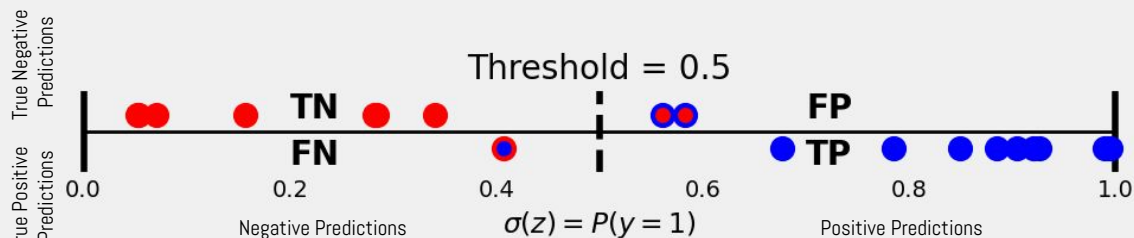
Classification Threshold



Confusion Matrix



$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$



$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$